Lab 15 — Praktijklab: een database die haar Pods overleeft

Doel: een volume vragen en het cluster het zien maken, erin schrijven vanuit twee opeenvolgende Pods, dan PostgreSQL uitrollen als StatefulSet met zijn headless Service, de Pod verwijderen, de StatefulSet verwijderen, naar nul replica's gaan — en telkens de gegevens terugvinden.

Vereisten — Lab 14 afgewerkt: k3s; de image `postgres:16-alpine` wordt door het cluster van Docker Hub gehaald.

Meegeleverde bestanden (`files/`) - `pvc-demo.yaml` — een PVC van 100 Mi en een Pod `writer` die erin schrijft. - `secret-db.yaml` — het wachtwoord van de database (lab 14). - `statefulset-db.yaml` — het headless Service `db` en de PostgreSQL-StatefulSet met `volumeClaimTemplates`.

Stap 1 — De StorageClass en een eerste PVC

```
mkdir -p ~/labo-docker/15 && cd ~/labo-docker/15 && cp <pad-naar-het-lab>/files/* .
kubectl create namespace lab15 ; kubectl config set-context --current --namespace=lab15
kubectl get storageclass
kubectl apply -f pvc-demo.yaml
sleep 3 ; kubectl get pvc
sleep 10 ; kubectl get pvc ; kubectl get pv
```

Observeer `local-path (default) rancher.io/local-path Delete WaitForFirstConsumer`; een PVC `demo-data` die `Pending` is als je snel bent, en dan `Bound` aan een volume `pvc-<uuid>` van `100Mi RWO`; en een PV met dezelfde naam, `RECLAIM POLICY Delete`, `CLAIM lab15/demo-data`.

```
PV=$(kubectl get pvc demo-data -o jsonpath='{.spec.volumeName}')
kubectl get pv $PV -o jsonpath='{.spec.local.path}{"\n"}
{.spec.nodeAffinity.required.nodeSelectorTerms[0].matchExpressions[0].values[0]}{"\n"}'
kubectl exec writer -- cat /data/journal.txt
sudo ls /var/lib/rancher/k3s/storage/
```

Observeer het pad `/var/lib/rancher/k3s/storage/pvc-<uuid>_lab15_demo-data`, de affiniteit naar jouw node, een regel `ecrit par writer le …` in het bestand, en de map van het volume op de schijf van de node.

Uitleg. Je vroeg 100 Mi; de StorageClass wachtte op de eerste Pod (`WaitForFirstConsumer`), maakte dan een map op de node aan en een PV gekoppeld aan je PVC. De Pod kent alleen de PVC.

Stap 2 — Het volume overleeft de Pod, niet de PVC

```
kubectl delete pod writer ; kubectl apply -f pvc-demo.yaml ; sleep 8
kubectl exec writer -- cat /data/journal.txt
kubectl delete pod writer ; kubectl delete pvc demo-data ; sleep 5 ; kubectl get pv | grep -c
demo-data
```

Observeer `persistentvolumeclaim/demo-data unchanged` en een nieuwe `writer` die **twee** regels leest: de zijne en die van de vorige Pod; daarna, na het verwijderen van de PVC, `0` : de PV (beleid `Delete`) is met de gegevens verdwenen.

Uitleg. De PVC houdt het volume vast; de Pods passeren. De PVC verwijderen activeert het terugwinbeleid.

Stap 3 — PostgreSQL als StatefulSet

```
kubectl apply -f secret-db.yaml -f statefulset-db.yaml
kubectl rollout status statefulset/db --timeout=120s
kubectl get statefulset,pods,pvc,svc
```

Observeer (een waarschuwing `spec.SessionAffinity is ignored for headless services` is zonder gevolg): `statefulset.apps/db 1/1` , een Pod met de naam **`db-0`** — geen hash —, een PVC **`data-db-0`** van `500Mi` gekoppeld, en een Service `db` met `CLUSTER-IP None` .

```
kubectl run client --image=docker.io/library/alpine --restart=Never --command -- sleep 600 ;
sleep 6
kubectl exec client -- getent hosts db db-0.db
kubectl exec client -- sh -c 'nc -zv db-0.db.lab15.svc.cluster.local 5432'
kubectl delete pod client --now
```

Observeer hetzelfde IP onder twee namen, `db.lab15.svc.cluster.local` en **`db-0.db.lab15.svc.cluster.local`** , en poort 5432 open: het headless Service geeft elke Pod een naam.

Uitleg. `volumeClaimTemplates` heeft `data-db-0` aangemaakt voor `db-0` ; het Service `clusterIP: None` verdeelt niet, het benoemt.

Stap 4 — De Pod breken, de database behouden

```
kubectl exec db-0 -- psql -U postgres -c "CREATE TABLE clients(id serial, nom text); INSERT
INTO clients(nom) VALUES ('Durand'),('Peeters');"
kubectl delete pod db-0 ; sleep 2 ; kubectl get pods
kubectl rollout status statefulset/db --timeout=120s
kubectl get pvc
kubectl exec db-0 -- psql -U postgres -c "SELECT * FROM clients;"
```

Observeer `CREATE TABLE` / `INSERT 0 2` ; na de verwijdering een Pod die **opnieuw** **`db-0`** heet (`AGE 2s`); dezelfde PVC `data-db-0` ; en de twee klanten teruggevonden.

```
kubectl scale statefulset db --replicas=0 ; sleep 10 ; kubectl get pods ; kubectl get pvc
kubectl scale statefulset db --replicas=1 ; kubectl rollout status statefulset/db --
timeout=120s
kubectl exec db-0 -- psql -U postgres -c "SELECT count(*) FROM clients;"
```

Observeer `No resources found` voor de Pods maar `data-db-0` nog altijd `Bound` bij nul replica's, en `count = 2` bij de terugkeer.

Uitleg. De StatefulSet koppelt de Pod `db-0` bij elke nieuwe aanmaak aan de PVC `data-db-0`; terugschalen naar nul geeft het volume niet vrij.

Stap 5 — Back-uppen, de StatefulSet verwijderen, opnieuw aanmaken

```
kubectl exec db-0 -- pg_dump -U postgres > dump.sql ; grep -A3 'COPY public.clients' dump.sql
kubectl delete statefulset db ; sleep 8 ; kubectl get pods ; kubectl get pvc
kubectl apply -f statefulset-db.yaml ; kubectl rollout status statefulset/db --timeout=120s
kubectl exec db-0 -- psql -U postgres -c "SELECT count(*) FROM clients;"
kubectl get pv
```

Observeer de `dump.sql` met de twee regels (logische back-up, terwijl de database draait, zoals in lab 06); na `delete statefulset` geen Pod meer maar `data-db-0` **nog altijd aanwezig**; na de nieuwe aanmaak `count = 2`; en een PV `Bound` aan `lab15/data-db-0`.

Uitleg. Een StatefulSet verwijderen verwijdert nooit zijn PVC's: de database wacht op de volgende StatefulSet met dezelfde naam.

Opruimen

```
kubectl delete namespace lab15 ; sleep 20
kubectl get pv | grep -c lab15
kubectl config set-context --current --namespace=default
```

Observeer `0`: het verwijderen van de namespace heeft de PVC verwijderd, dus de PV en de gegevens (beleid `Delete`). Dat is de enige handeling in dit lab die de database werkelijk vernietigt. Hou `dump.sql`: dat is je back-up.

Wat je nu moet kunnen beweren

- `local-path`, `WaitForFirstConsumer`, `Delete`: een PVC `Pending` tot de eerste Pod, een PV `pvc-<uuid>` in `/var/lib/rancher/k3s/storage/`, met node-affiniteit.
- Twee opeenvolgende Pods lezen dezelfde `journal.txt`; de PVC verwijderen verwijdert de PV.
- StatefulSet: `db-0`, `data-db-0`, Service `CLUSTER-IP None`, `db-0.db.lab15.svc.cluster.local`.
- Pod verwijderd, scale naar 0, StatefulSet verwijderd: `data-db-0` blijft, `count = 2` bij elke terugkeer.
- Namespace verwijderd: PV en gegevens weg; `pg_dump` via `kubectl exec` is de back-up.